



KAN vs MLP

Как устроено и зачем

Почему MLP — это «чёрный ящик»?

Многослойный персептрон (MLP) — основа современного глубокого обучения:

- Фиксированные функции активации (ReLU, SiLU) в **узлах**
- Обучаются только **скалярные веса** на рёбрах
- Выразительная сила гарантирована универсальной теоремой аппроксимации

Проблема:

- MLP — это закрытый «чёрный ящик»
- Мы не можем понять, как именно сеть пришла к решению
- Это критично для научных задач, где важна интерпретируемость

Идея KAN: вывернуть MLP наизнанку

KAN основаны на **теореме Колмогорова–Арнольда** (1957):

любая непрерывная функция f от n переменных на компакте может быть представлена в виде композиции одномерных функций и операций сложения:

$$f(x) = \sum_{q=1}^{2^{n+1}} \Phi_q \left(\sum_{p=1}^n \phi_{q,p}(x_p) \right),$$

Ключевая идея: функции активации находятся **на рёбрах**, а не в узлах.

Параметры	MLP	KAN
Активации	В узлах (фиксированные)	На рёбрах (обучаемые)
Веса	Скалярные	Одномерные функции
Узлы	Вычисляют активации	Только суммируют

MLP vs KAN: сравнение архитектур

MLP:

- Нейроны = фиксированные функции активации
- Рёбра = скалярные веса
- Граф вычислений: активация → взвешивание → сумма

KAN:

- Нейроны = сумматоры
- Рёбра = обучаемые функции $\phi(x)$
- Граф вычислений: $x \rightarrow \phi(x) \rightarrow$ сумма
- Слой: $x_{l+1,j} = \sum_{i=1}^{n_{in}} \phi_{l,j,i}(x_{l,i})$

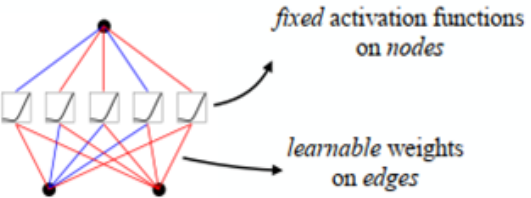
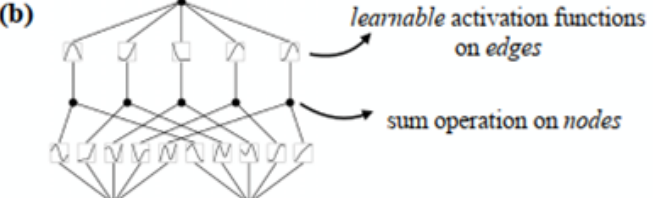
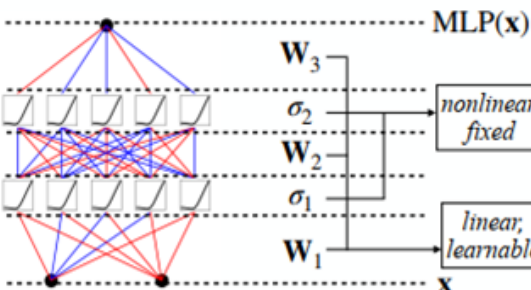
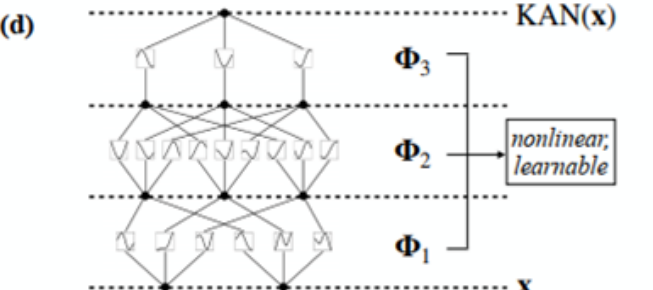
Model	Multi-Layer Perceptron (MLP)	Kolmogorov-Arnold Network (KAN)
Theorem	Universal Approximation Theorem	Kolmogorov-Arnold Representation Theorem
Formula (Shallow)	$f(\mathbf{x}) \approx \sum_{i=1}^{N(\epsilon)} a_i \sigma(\mathbf{w}_i \cdot \mathbf{x} + b_i)$	$f(\mathbf{x}) = \sum_{q=1}^{2n+1} \Phi_q \left(\sum_{p=1}^n \phi_{q,p}(x_p) \right)$
Model (Shallow)		
Formula (Deep)	$\text{MLP}(\mathbf{x}) = (\mathbf{W}_3 \circ \sigma_2 \circ \mathbf{W}_2 \circ \sigma_1 \circ \mathbf{W}_1)(\mathbf{x})$	$\text{KAN}(\mathbf{x}) = (\Phi_3 \circ \Phi_2 \circ \Phi_1)(\mathbf{x})$
Model (Deep)		

Рисунок 1: Многослойные персептроны (MLP) против сетей Колмогорова-Арнольда (KAN)

Обучаемая функция на ребре

Каждая функция активации представляется как:

$$\phi(x) = \omega_b b(x) + \omega_s \text{spline}(x),$$

$$\text{spline}(x) = \sum_i c_i B_i(x)$$

Компоненты:

$b(x)$ — базисная функция (обычно $\text{silu}(x) = x \cdot \sigma(x)$);

$B_i(x)$ — В-сплайновые базисные функции порядка k (кубические, $k=3$);

c_i — обучаемые коэффициенты сплайна;

ω_b, ω_s — обучаемые масштабирующие коэффициенты.

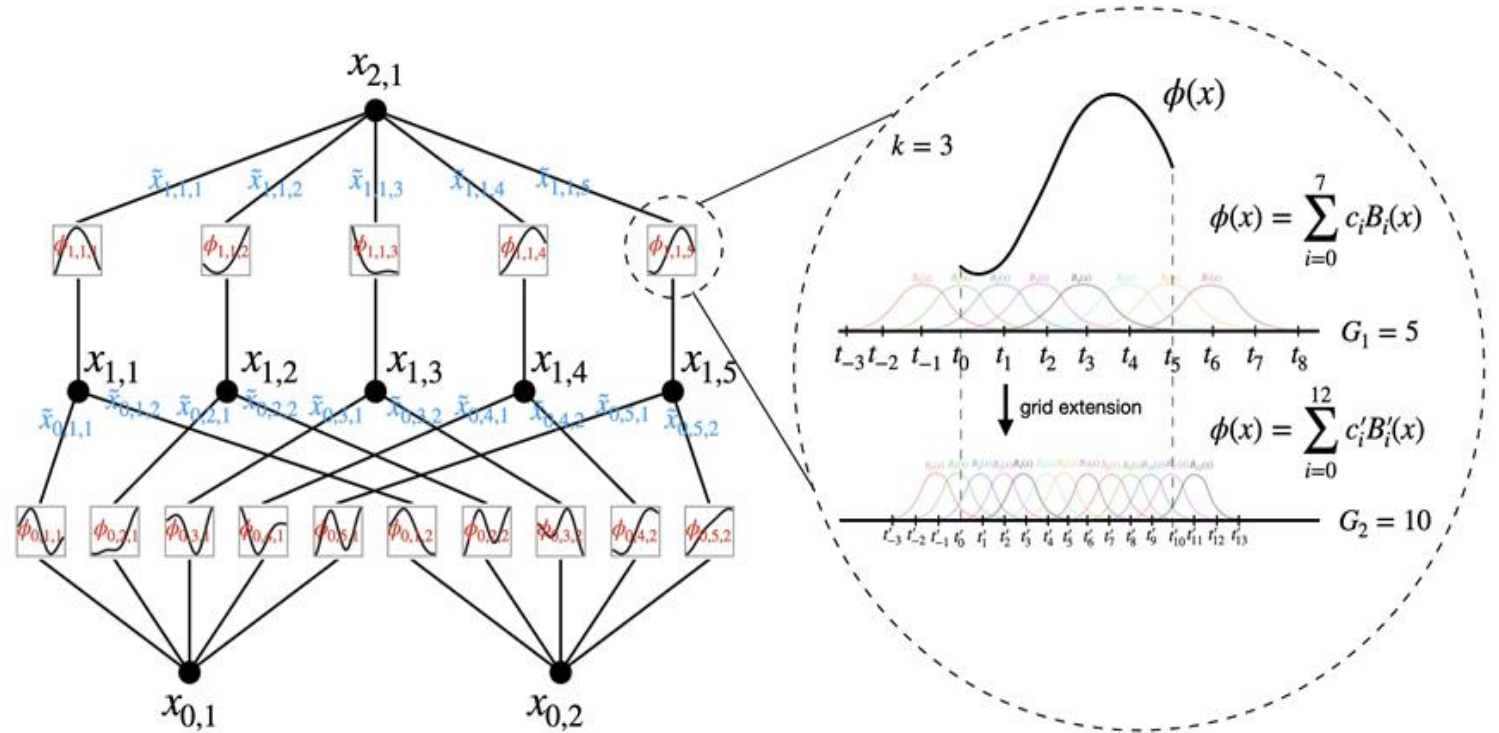


Рисунок 2: Слева: Обозначения активаций, проходящих через сеть.

Справа: функция активации параметризована как В-сплайн, что позволяет переключаться между крупнозернистыми и мелкозернистыми сетками.

Обучение KAN vs MLP — ключевые различия

Базовый процесс

	MLP	KAN
Алгоритм	Стандартный backpropagation	Backpropagation + дополнительные шаги
Оптимизаторы	SGD, Adam (стабильно)	Adam, LBFGS (при $G > 100$ LBFGS замедляется)
Градиенты	По скалярным весам w	По коэффициентам сплайнов c_i и масштабам w_b, w_s
Регуляризация	L1/L2, dropout	L1 на сплайновых коэффициентах (разреживание сети)

Специфические шаги обучения KAN

1. Динамическое обновление сетки (обязательно)

- В-сплайны определены на фиксированной сетке $[a, b]$
- Входные значения могут выходить за пределы $\rightarrow$ нестабильность
- Решение: периодическая **перестройка сетки** под текущее распределение данных

2. Расширение сетки - увеличение числа интервалов сетки $G \rightarrow G'$ для повышения точности

3. Чувствительность к гиперпараметрам

- Требуется настройка: порядок сплайна k (обычно 3), число интервалов G , частота обновления
- Шум $\rightarrow$ осцилляции сплайнов $\rightarrow$ необходима регуляризация

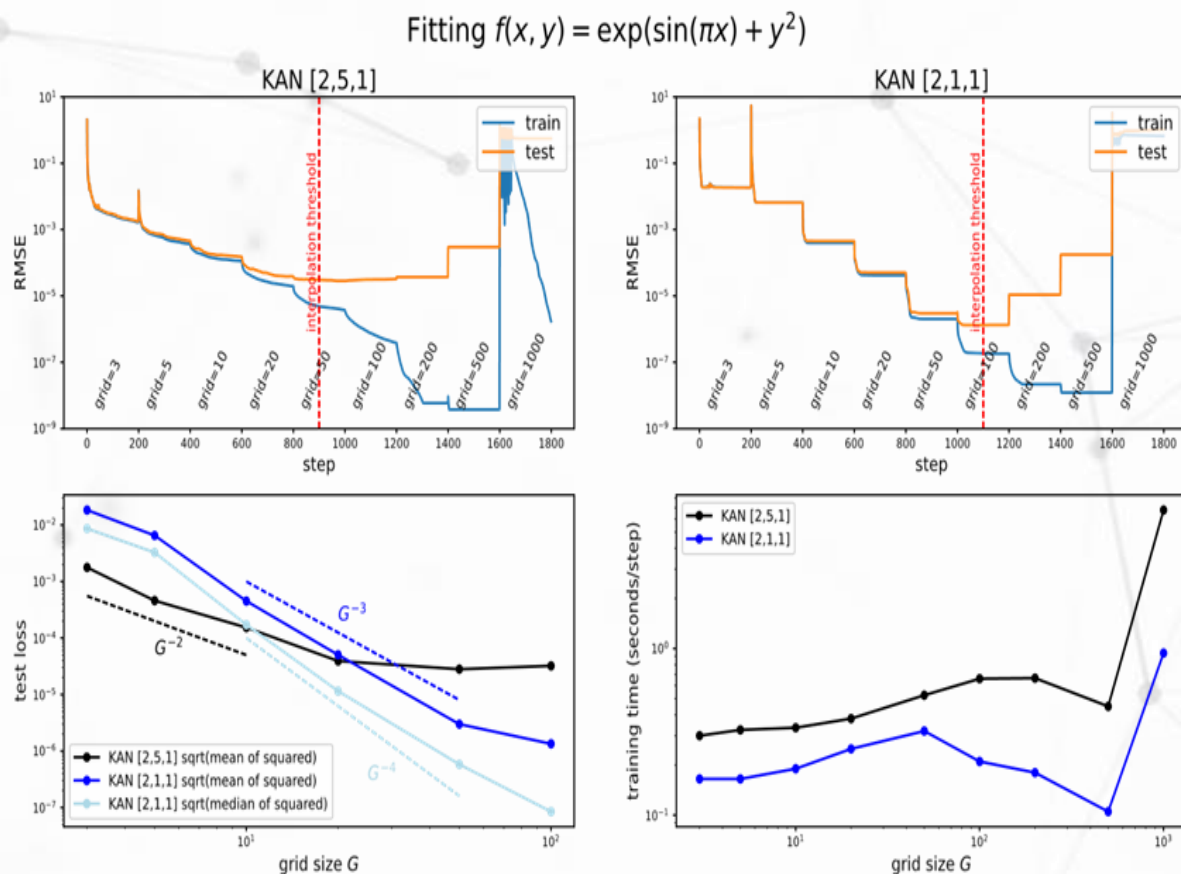
Ключевое отличие №1 — Расширение сетки

Как KAN повышает точность без переобучения

В MLP: хотите повысить точность → увеличиваете ширину/глубину → **переобучаете с нуля** (дорого)

В KAN: есть процедура Grid Extension:

1. Обучаете сеть на грубой сетке (G интервалов)
2. Строите мелкую сетку ($G' > G$ интервалов)
3. Инициализируете коэффициенты новой сетки через наименьшие квадраты
4. Дообучаете — точность растёт **без переобучения с нуля**



Теоретическая оценка: $\|f - \text{KAN}_G\| \leq C \cdot G^{-k}$
Для кубических сплайнов ($k=3$): ошибка убывает как G^{-3} — очень быстро!

Рисунок 3: Вверху: динамика обучения KAN [2,5,1] ([2,1,1]) с расширением сетки. Внизу слева: тест RMSE следует законам масштабирования в зависимости от размера сетки G . Внизу справа: время обучения благоприятно масштабируется с размером сетки G .

Ключевое отличие №2 — Интерпретируемость

KAN — это «белый ящик»

MLP даёт ответ, но не объяснение.

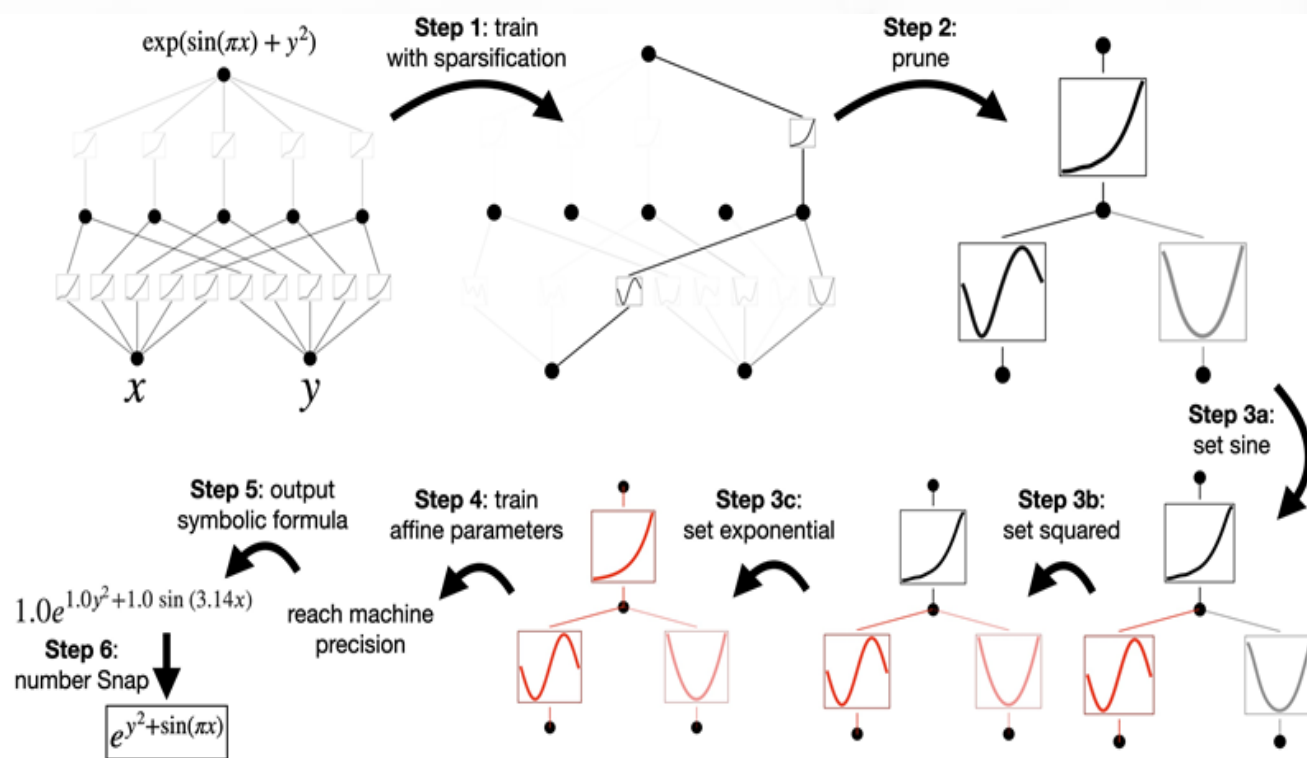


Рисунок 4: Пример символической регрессии с помощью KAN.

KAN можно:

- **Визуализировать**: каждое ребро — это функция, её можно нарисовать
- **Анализировать**: определить важность каждого признака
- **Извлекать формулы**: превратить сеть в символическое выражение

Пример: обучение на $f(x,y)=\sin(x)+ey$

- KAN выделит одно ребро для $\sin(x)$, другое для ey
- Мы увидим это в визуализации

Для науки это революция: ИИ не просто предсказывает, а **открывает законы**

Сравнение параметров и сложности

Нюансы:

- KAN требуют **больше параметров** при одинаковой ширине
- KAN используют **значительно меньшую ширину** ($N \approx 10$ вместо $N \approx 100$)
- Это **компенсирует** разницу

Скорость:

- KAN медленнее MLP в **10–100 раз** из-за B-сплайнов
- Сложно распараллелить

Параметр	MLP	KAN
Параметро в на слой	$O(N^2)$	$O(N^2G)$
Глубина	L	L
Итого	$O(N^2L)$	$O(N^2LG)$

Где KAN побеждает MLP

Области превосходства

- ✓ **Символьная регрессия** - восстановление математических формул из данных
- ✓ **Научные вычисления (AI+Science)** - физика, химия, биология, открытие законов, сохраняющихся величин
- ✓ **Решение дифференциальных уравнений (PDE)** - более быстрые законы масштабирования
- ✓ **Табличные данные с небольшим числом признаков** - превосходная точность при малом размере сети
- ✓ **Интерпретируемые модели** - возможность объяснить решение

Где MLP остаётся лучше

Ограничения KAN

- ✗ **Крупные сенсорные данные** (изображения, видео, аудио) MLP / CNN / Transformer лучше
- ✗ **Обработка естественного языка (NLP)** - тексты — неструктурированные данные
- ✗ **Классическое машинное обучение с большим числом признаков** - MLP проще масштабировать
- ✗ **Индустриальные приложения общего назначения** - MLP быстрее, дешевле, проще в развёртывании
- ✗ **Глубокие сети ($L > 5$)** - KAN не стабильно превосходят MLP, проблемы с осцилляциями сплайнов

Библиотека ruKan: AI + Science

Три новых компонента:

- **MultKAN** — узлы умножения
 - Выявление мультипликативных структур ($y=x \cdot y$)
 - Нет обучаемых параметров в узлах умножения
- **kanpiler** — компилятор формул
 - Превращает символьную формулу в KAN
 - Парсинг → модификация → сборка
- **Tree converter** — преобразование в деревья
 - Строит интерпретируемое дерево решений
 - Открывает лагранжианы, симметрии, законы

+ Оптимизации:

- Память: $O(N^2 LG) \rightarrow O(L \cdot N \cdot G)$
- Поддержка GPU
- Ускорение в сотни раз

Двунаправленная синергия

Science → KAN (встраивание знаний)

Уровень	Инструмент	Пользователь задает
Важные признаки	augment_input	«у зависит только от x_1 , x_2 »
Модульные структуры	module	« x_1 и x_2 аддитивно влияют на y »
Точная формула	kanpiler	« $y = \sin(x_1) + \exp(x_2)$ »

KAN → Science (извлечение знаний)

Инструмент	Что даёт
prune_input	Определяет важные признаки
auto_swap	Выявляет модули
auto_symbolic + symbolic_formula	Извлекает формулы
rewind	Проверяет гипотезы

Подведение итогов

Зачем нужен KAN?

KAN — это мощный нишевый инструмент для:

Научных исследований

- Интерпретируемость
- Открытие законов
- Извлечение формул

Задач со структурными данными

- Гладкие композиционные функции
- Мало признаков
- Требуется точность

AI+Science:

- Симбиоз ИИ и науки
- Двухнаправленный обмен знаниями

KAN — это НЕ замена MLP

Для промышленных задач общего назначения → MLP остаётся выбором №1